ВВЕДЕНИЕ

Теперь, когда вы имеете представление о языке JavaScript, давайте рассмотрим некоторые способы его применения в современном веб-дизайне. Сначала мы рассмотрим DOM-скриптинг, позволяющий манипулировать элементами, атрибутами и текстом на странице.

ЗНАКОМСТВО С DOM

В этом курсе вы несколько раз встречали упоминания объектной модели документа (DOM), но сейчас самое время уделить ей должное внимание. DOM позволяет получать доступ к содержимому документа и управлять им. Мы обычно используем её для HTML, но DOM можно использовать и с любым языком XML. И хотя мы сосредоточены на его связи с JavaScript, стоит отметить, что к DOM могут обращаться и другие языки, такие как PHP, Ruby, C++ и другие.

Хотя DOM уровня 1 был выпущен W3C в 1998 году, DOM-скриптинг начал набирать обороты почти пять лет спустя.

DOM — это программный интерфейс (API) для HTML- и XML-страниц. Он предоставляет структурированную карту документа, а также набор методов для взаимодействия с элементами, содержащимися в нём. По сути, он преобразует нашу разметку в формат, понятный JavaScript (и другим языкам программирования). Основная суть заключается в том, что DOM служит картой всех элементов на странице и позволяет нам выполнять с ними различные действия. Мы можем использовать её для поиска элементов по их именам или атрибутам, а затем добавлять, изменять или удалять элементы и их содержимое.

Без DOM JavaScript не имел бы никакого представления о содержимом документа, и под этим я подразумеваю всё его содержимое. Всё, от doctype страницы до каждой отдельной буквы в тексте, доступно через DOM и обрабатывается с помощью JavaScript.

Дерево узлов

DOM проще всего представить себе в виде дерева документа, представленного на РИСУНКЕ 1. Вы видели документы, представленные в таком виде, когда изучали CSS-селекторы.

```
<!DOCTYPE html>
<html>
<head>
<title>Заголовок документа</title>
<meta charset="utf-8" />
</head>
<body>
<div>
<h1>Заголовок</h1>
<p>Текст абзаца со ссылкой <a href="foo.html"></a>.</p>
</div>
<div>
<p>Дополнительный текст.</p>
</div>
```

```
</body>
</html>
```

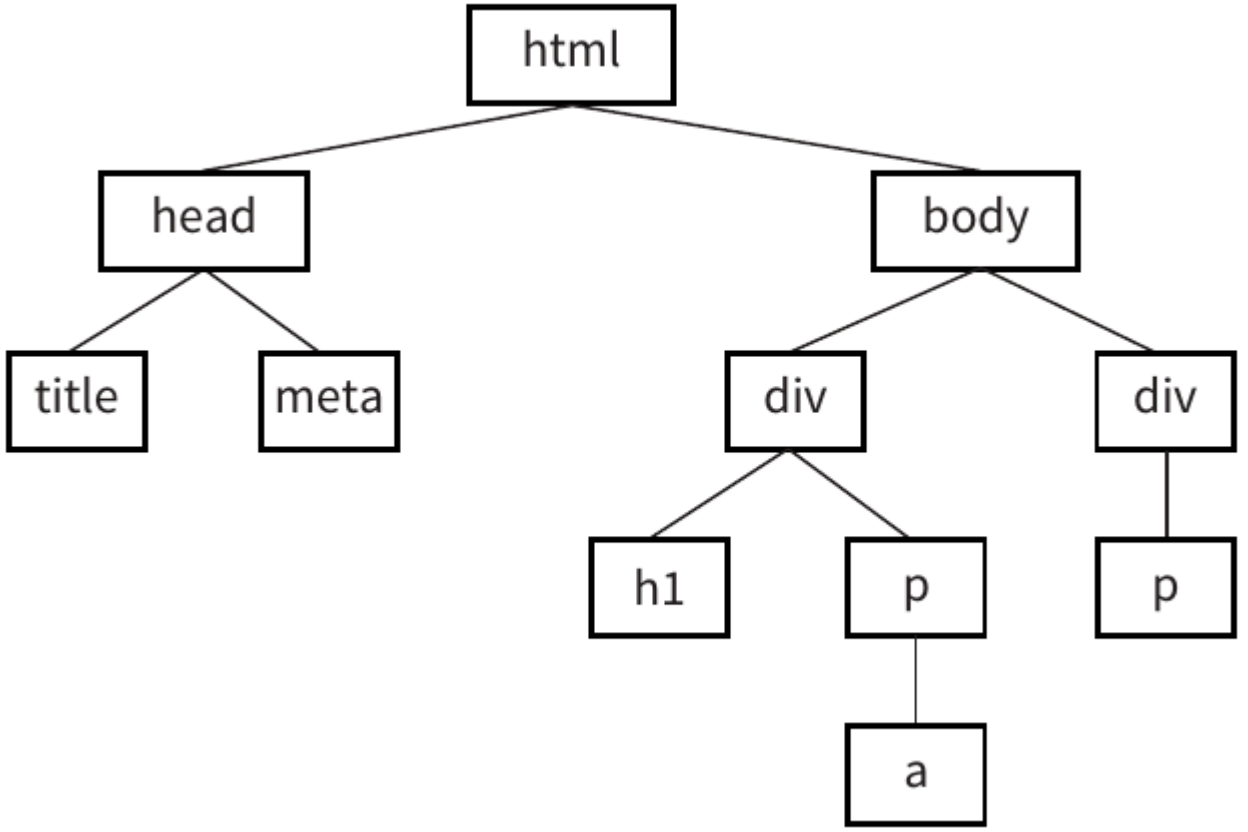


Рисунок 1

Каждый элемент на странице называется узлом. Если представить DOM как дерево, каждый узел — это отдельная ветвь, которая может содержать дополнительные ветви. Но DOM обеспечивает более глубокий доступ к контенту, чем CSS, поскольку он также рассматривает сам контент как узел. На рисунке 2 показана структура первого элемента p. Элемент, его атрибуты и его содержимое — это узлы в дереве узлов DOM.

Paragraph text with a link here.

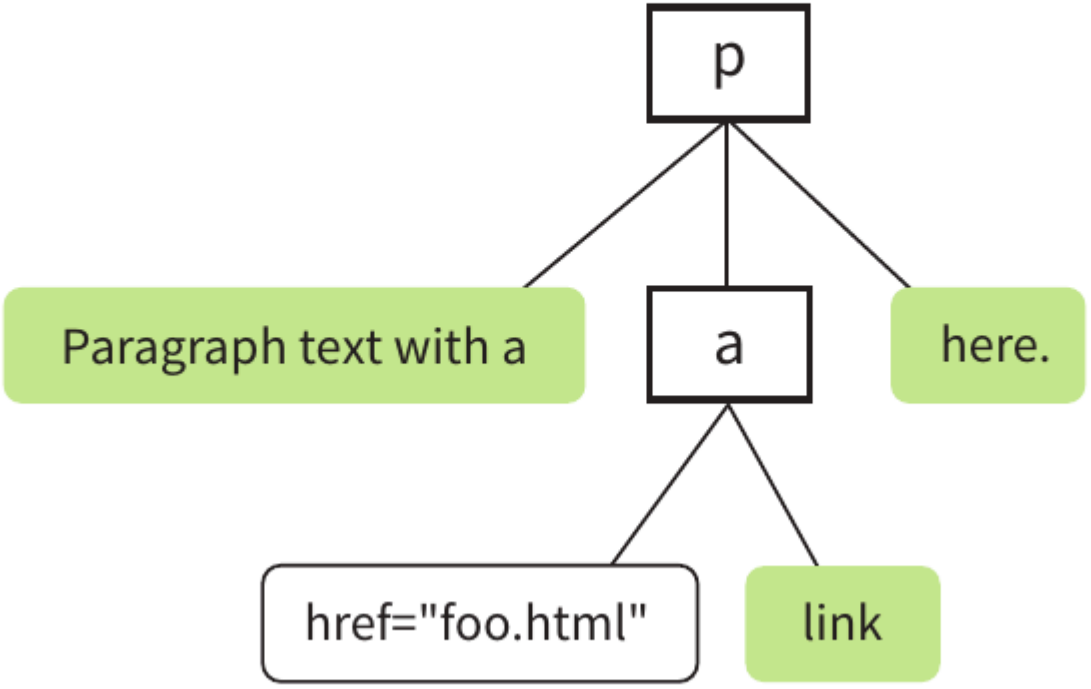


Рисунок 2

DOM также предоставляет стандартизированный набор методов и функций, с помощью которых JavaScript может взаимодействовать с элементами страницы. Большая часть сценариев DOM включает

чтение и запись данных в документ. Существует несколько способов использования DOM для поиска нужной информации в документе. Давайте рассмотрим некоторые методы доступа к объектам, определённым DOM, а также методы управления этими элементами.

Доступ к узлам DOM

Объект документа в DOM идентифицирует саму страницу и чаще всего служит отправной точкой для сканирования DOM. Объект документа имеет ряд стандартных свойств и методов для доступа к коллекциям элементов. Это напоминает свойство длины, о котором мы говорили на предыдущих лекциях. Подобно тому, как длина является стандартным свойством всех массивов, объект документа имеет ряд встроенных свойств, содержащих информацию о документе. Затем мы переходим к нужному элементу, объединяя эти свойства и методы в цепочку, разделяя их точками, чтобы сформировать своего рода маршрут по документу.

Чтобы дать вам общее представление об этом, оператор в этом примере предлагает просмотреть страницу (документ), найти элемент со значением идентификатора «beginner», найти HTML-содержимое этого элемента (innerHTML) и сохранить его в переменной (foo):

```
var foo = document.getElementById("beginner").innerHTML;
```

Поскольку цепочки, как правило, длинные, часто каждое свойство или метод разбивается на отдельную строку для удобства чтения.

Помните, что пробелы в JavaScript игнорируются, поэтому они не влияют на разбор оператора.


```
var foo = document.getElementById("beginner").innerHTML;
```


Существует несколько методов доступа к узлам в документе.

DOM также предоставляет стандартизированный набор методов и функций, с помощью которых JavaScript может взаимодействовать с

элементами страниц. Большая часть влияния DOM включается и читается запись данных в документ. Существует несколько способов использования DOM для поиска необходимой информации в документе. Давайте рассмотрим некоторые методы доступа к объектам, определенным DOM, а также методы управления древними элементами.

Доступ к узлам DOM

Объектный документ в DOM идентифицирует саму страницу и чаще всего служит отправной точкой для внешнего DOM. Объектный документ содержит ряд стандартных свойств и методов для доступа к элементам коллекции. Это напоминает свойство длины, о котором мы говорили на предыдущих лекциях. Учитывая то, что длина является стандартным свойством всех массивов, объектный документ имеет ряд встроенных свойств, содержащих информацию о документе. Затем мы переходим к нужному элементу, объединяем эти

свойства и методы в цепочку, разделяя их точками, чтобы применить свой вид маршрута к документу.

Чтобы дать вам общее представление об этом, оператор в этом примере предлагает просмотреть страницу (документ), найти элемент со значением идентификатора «beginner», найти HTML-содержимое этого элемента (innerHTML) и сохранить его в переменной (foo):

```
var foo = document.getElementById("beginner").innerHTML;
```

Поскольку цепочки, как правило, длинные, часто каждое свойство или метод разбивается на соответствующий текст для удобства чтения.

Помните, что пробелы в JavaScript игнорируются, поэтому они не влияют на операцию разбора.


```
var foo = document.getElementById("beginner").innerHTML;
```

Существует несколько способов доступа к узлам в документе.

```
<img src="photo.jpg" alt="" id="lead-photo" />
```

Мы включаем значение идентификатора в качестве аргумента метода getElementById():

```
var photo = document.getElementById("lead-photo");
```

По значению атрибута класса: `getElementsByClassName()`

Это позволяет получать доступ к узлам документа на основе значения атрибута класса. Этот оператор присваивает любой элемент со значением класса «column-a» переменной firstColumn, что позволяет легко обращаться к нему из скрипта:

```
var firstColumn = document.getElementsByClassName("column-a");
```

Как и getElementsByTagName(), этот метод возвращает список узлов, к которому можно обращаться по индексу или перебирать их по одному.

По селектору: `querySelectorAll()`

`querySelectorAll()` позволяет получить доступ к узлам DOM на основе селектора CSSstyle. Синтаксис аргументов в следующих примерах должен быть вам знаком. Это может быть так же просто, как доступ к дочерним элементам определенного элемента:

```
var sidebarPara = document.querySelectorAll(".sidebar p");
```

или такой же сложный, как выбор элемента на основе атрибута:

```
var textInput = document.querySelectorAll("input[type='text']");
```

`querySelectorAll()` возвращает список узлов, например `getElementsByTagName()` и `getElementsByClassName()`, даже если

селектор соответствует только одному элементу.

Доступ к значению атрибута: `getAttribute()`

Как я уже упоминал ранее, элементы — не единственное, к чему можно получить доступ с помощью DOM. Чтобы получить значение атрибута, прикреплённого к узлу элемента, мы вызываем getAttribute() с одним аргументом: именем атрибута. Предположим, у нас есть изображение stratocaster.jpg, размеченное следующим образом:

```
<img src="stratocaster.jpg" alt="electric guitar" id="lead-image" />
```

В следующем примере мы получаем доступ к этому изображению (getElementbyId()) и сохраняем ссылку на него в переменной ("bigImage"). После этого мы можем получить доступ к любому атрибуту элемента (alt, src или id), указав его в качестве аргумента в методе getAttribute(). В этом примере мы получаем значение атрибута src и используем его в качестве содержимого в

оповещении. (Не понимаю, зачем это вообще нужно, но метод здесь продемонстрирован.)

```
var bigImage = document.getElementById("lead-image");
alert(bigImage.getAttribute("src")); // Оповещения
"stratocaster.jpg".
```

Управление узлами

После того, как мы получили доступ к узлу одним из рассмотренных ранее методов, DOM предоставляет нам несколько встроенных методов для управления этими элементами, их атрибутами и содержимым.

setAttribute()

Продолжаем предыдущий пример: мы видели, как получить значение атрибута, но что, если мы хотим установить значение атрибута src на новый путь? Используйте setAttribute()! Этот метод требует два

аргумента: атрибут, который нужно изменить, и новое значение этого атрибута.

В этом примере мы используем фрагмент JavaScript для замены изображения, изменив значение атрибута src:

```
var bigImage = document.getElementById("lead-image");
bigImage.setAttribute("src", "lespaul.jpg");
```

Только подумайте, сколько всего можно сделать с документом, изменив значения атрибутов. Здесь мы заменили изображение, но этот же метод можно использовать для внесения ряда изменений в документ:

Обновить атрибуты «check» флажков и переключателей в зависимости от взаимодействия пользователя с другими элементами страницы.

Найти элемент link для нашего файла .css и указать значение href на другую таблицу стилей, изменив все стили страницы.

Обновить атрибут title, добавив информацию о состоянии элемента (например, «этот элемент в данный момент выбран»).

innerHTML

innerHTML предоставляет нам простой метод для доступа к тексту и разметке внутри элемента и их изменения. Он работает иначе, чем методы, которые мы рассматривали ранее. Допустим, нам нужен быстрый способ добавить абзац текста к первому элементу на странице с классом intro:

```
var introDiv = document.getElementsByClassName("intro");
introDiv[0].innerHTML = "<p>Это наш вступительный текст</p>";
```

Второй оператор здесь добавляет содержимое строки в introDiv (элемент со значением класса "intro") как в реальный рабочий элемент, поскольку innerHTML сообщает JavaScript о необходимости обработки строк "`<p>`" и "`</p>`" как разметки.

style

DOM также позволяет добавлять, изменять или удалять стиль CSS из элемента с помощью свойства style. Это работает аналогично применению стиля с помощью встроенного атрибута style. Отдельные свойства CSS доступны как свойства свойства style. Уверен, вы сможете понять, что делают эти операторы, используя свои новые знания CSS и DOM:

```
document.getElementById("intro").style.color = "#fff";

document.getElementById("intro").style.backgroundColor = "#f58220";
//orange
```

В JavaScript и DOM имена свойств, которые в CSS пишутся через дефис (например, background-color и border-top-width), преобразуются в «верблюжий» регистр (backgroundColor и borderTopWidth соответственно), чтобы символ «–» не воспринимался как оператор.

В только что рассмотренных вами примерах свойство style используется для установки стилей узла. Его также можно использовать для получения значения стиля для использования в других местах скрипта. Этот оператор получает цвет фона элемента #intro и присваивает его переменной brandColor:

```
var brandColor =
document.getElementById("intro").style.backgroundColor;
```

Добавление и удаление элементов

До сих пор мы рассматривали примеры получения и установки узлов в существующем документе. DOM также позволяет разработчикам изменять структуру документа, добавляя и удаляя узлы «на лету». Начнём с создания новых узлов, что довольно просто, а затем посмотрим, как добавлять созданные узлы на страницу. Представленные здесь методы более хирургичны и точны, чем

добавление контента с помощью innerHTML. Заодно мы удалим и узлы.

createElement()

Чтобы создать новый элемент, используйте метод с удачным названием createElement(). Эта функция принимает один аргумент: создаваемый элемент. Использование этого метода на первый взгляд может показаться несколько нелогичным, поскольку новый элемент не появляется на странице сразу. После создания элемента таким образом он остаётся «плавающим в эфире» JavaScript до тех пор, пока мы не добавим его в документ. Представьте это как создание ссылки на новый элемент, который находится исключительно в памяти — то, чем мы можем манипулировать в JavaScript по своему усмотрению, а затем добавить на страницу, когда будем готовы:

```
var newDiv = document.createElement("div");
```

createTextNode()

Если мы хотим ввести текст в созданный нами или существующий элемент на странице, мы можем вызвать метод createTextNode(). Чтобы использовать его, передайте строку текста в качестве аргумента, и метод создаст версию этого текста, адаптированную для DOM, готовую для размещения на странице. Как и createElement(), этот метод создаёт ссылку на новый текстовый узел, который мы можем сохранить в переменной и добавить на страницу, когда это потребуется:

```
var ourText = document.createTextNode("This is our text.");
```

appendChild()

Итак, мы создали новый элемент и новую строку текста, но как сделать их частью документа? Перейдем к методу appendChild(). Этот метод принимает один аргумент: узел, который вы хотите добавить в DOM. Вызывается для существующего элемента,

который будет его родительским в структуре документа. Пришло время для примера.

Вот простой div на странице с идентификатором «our-div»:

```
<div id="our-div"></div>
```

Допустим, мы хотим добавить абзац в #our-div с текстом «Hello, world!». Начнем с создания элемента p (document.createElement()), а также текстового узла для содержимого, которое будет в нем находиться (createTextNode()):

```
var ourDiv = document.getElementById("our-div");
var newParagraph = document.createElement("p");

var copy = document.createTextNode("Hello, world!");
```

Теперь у нас есть элемент и текст, и мы можем использовать appendChild(), чтобы собрать всё воедино:

```
newParagraph.appendChild(copy);
ourDiv.appendChild(newParagraph);
```

Первый оператор добавляет копию (это наш текстовый узел «Hello, world!») к созданному нами новому абзацу (newParagraph), так что теперь у этого элемента есть контент. Вторая строка добавляет новый абзац к исходному div (ourDiv). Теперь ourDiv не будет пустовать в DOM, а отобразится на странице с текстом «Hello, world!». Вы, должно быть, поняли, как это работает. Как насчёт ещё парочки?

insertBefore()

Метод insertBefore(), как вы могли догадаться, вставляет элемент перед другим элементом. Он принимает два аргумента: первый — это вставляемый узел, а второй — элемент, перед которым он вставляется. Вам также необходимо знать родительский элемент, к которому будет добавлен элемент.

Так, например, чтобы вставить новый заголовок перед абзацем в этой разметке,

```
<div id="our-div">
<p id="our-paragraph">Текст нашего абзаца</p>
</div>
```

Мы сначала присваиваем имена переменных тегу div и содержащемуся в нем тегу p, а затем создаём элемент h1 и его текстовый узел и объединяем их, как в предыдущем примере:

```
var ourDiv = document.getElementById("our-div");
var para = document.getElementById("our-paragraph");
var newHeading = document.createElement("h1");
var headingText = document.createTextNode("Новый заголовок");
newHeading.appendChild(headingText); // Добавляем новый текстовый
узел к новому заголовку
```

Наконец, в последнем показанном здесь операторе метод insertBefore() помещает элемент newHeading h1 перед элементом para внутри ourDiv.

```
ourDiv.insertBefore(newHeading, para);
```

replaceChild()

Метод replaceChild() заменяет один узел другим и принимает два аргумента. Первый аргумент — это новый дочерний элемент (т.е. узел, который должен получиться в результате). Второй — это узел, который заменяется первым. Как и в случае с insertBefore(), необходимо также указать родительский элемент, в котором происходит замена. Для простоты предположим, что мы начинаем со следующей разметки:

```
<div id="our-div">
<div id="swap-me"></div>
```

```
</div>
```

И мы хотим заменить div с идентификатором «swap-me» изображением. Сначала мы создаём новый элемент img и устанавливаем атрибут src на путь к файлу изображения. В заключительном операторе мы используем replaceChild(), чтобы заменить swapMe на newImg.

```
var ourDiv = document.getElementById("our-div");
var swapMe = document.getElementById("swap-me");
var newImg = document.createElement("img"); // Создать новый элемент изображения
newImg.setAttribute("src", "path/to/image.jpg"); // Присвоить новому изображению атрибут "src"
ourDiv.replaceChild(newImg, swapMe);
removeChild();
```

Метод removeChild() удаляет узел или целую ветвь из дерева документа. Метод принимает один аргумент — имя удаляемого узла.

Помните, что DOM мыслит в терминах узлов, а не только элементов, поэтому дочерним элементом элемента может быть содержащийся в нём текст (узел), а не только другие элементы.

Как и appendChild(), метод removeChild() всегда вызывается для родительского элемента удаляемого элемента (отсюда и название «удалить дочерний элемент»). Это означает, что нам потребуется ссылка как на родительский узел, так и на удаляемый узел. Предположим, используется следующий шаблон разметки:

```
<div id="parent">
<div id="remove-me">
<p>Фу, мне здесь всё равно никогда не нравилось.</p>
</div>
</div>
```

Наш скрипт будет выглядеть примерно так:

```
var parentDiv = document.getElementById("parent");
var removeMe = document.getElementById("remove-me");
```

```
parentDiv.removeChild(removeMe); // Удаляет div с идентификатором "remove-me" со страницы.
```